

1. Claude Code

Claude Code를 어디에 사용할 것인가

Claude Code는 코드를 자동으로 완성해 주는 도구가 아닙니다. 3일차 수업에서 Claude Code는 기존 프로젝트 구조를 빠르게 이해하고, Agent 아키텍처 설계 관점에서 리뷰하는 보조 도구로 사용됩니다. 이 차이를 명확히 이해하는 것이 이번 장의 출발점입니다.

src/day3 폴더에는 제조 DB 조회, 기술 문서 검색, Tool 등록, Multi-Agent 역할 분담, 실행 기록 생성과 관련된 파일들이 함께 배치되어 있습니다. 코드를 실행하기 전에, 각 파일이 전체 Agent 흐름에서 어떤 책임을 갖는지 먼저 파악하는 것이 중요합니다.

설비 반복 알람 분석 Agent 예시

반복 알람 원인을 확인하는 Agent를 가정하면, Claude Code는 다음과 같이 구조를 정리하는 데 도움을 줄 수 있습니다.

- `postgres_db_tool.py` — 제조 DB와 설비 로그 조회 담당
- `search_manual_tool.py` — 기술 문서 검색 담당
- `multi_agent_roles.py` — Agent별 역할과 State 전달 구조 담당

Claude Code로 함께 검토할 수 있는 설계 항목

- MCP Tool 분할 기준 — 어떤 기능을 별도 Tool로 나눌 것인가
- Tool 설명과 입력·출력 구조 — LLM이 Tool을 올바르게 선택하도록 설계
- LangGraph Routing 흐름 — Tool 호출 조건과 분기 구조
- State와 `agent_steps`로 남겨야 할 실행 기록 항목

 중요한 것은 Claude Code의 답변을 그대로 따르는 것이 아니라, 사람이 제조 현업 적용 기준과 설계 기준을 가지고 검토하는 것입니다.

Claude Code 사용 시 반드시 지킬 원칙

Claude Code를 사용할 때 가장 먼저 해야 할 일은 코드를 수정하게 하는 것이 아니라 현재 구조를 설명하게 하는 것입니다. 수정 요청을 하기 전에 반드시 다음 세 가지를 먼저 확인해야 합니다. 해당 파일이 전체 Agent 흐름에서 어떤 위치에 있는지, 어떤 입력을 받아 어떤 출력을 반환하는지, 어떤 Tool이나 State와 연결되는지입니다. 구조를 이해하지 않은 상태에서 바로 수정을 요청하면, 기존 실습 흐름과 맞지 않는 방향으로 파일 구조나 함수 구조가 바뀔 수 있습니다.

원칙 1 — 구조 이해 먼저, 수정 요청은 나중에

Claude Code에게 먼저 "이 파일이 전체 흐름에서 어떤 역할을 하는가"를 설명하게 합니다. 파일 간 연결 구조, Tool 호출 조건, State 전달 경로를 파악한 뒤에 수정 여부를 판단합니다. 설계 리뷰 없이 바로 수정을 요청하는 것은 피해야 합니다.

원칙 2 — 수정 범위를 명확히 제한한다

코드 수정을 요청할 때는 기존 파일명, 함수명, 클래스명, 입력·출력 구조를 유지하도록 조건을 제시합니다. 전체 재작성보다 필요한 최소 변경안을 먼저 제안하게 해야 합니다. 예를 들어 "기존 함수 구조는 유지하고, 로그 출력 부분만 보강하는 최소 변경안을 제안해 달라"는 방식으로 요청 범위를 좁힙니다.

원칙 3 — 민감 정보는 입력하지 않는다

제조 DB 접속 정보, 실제 설비 로그, API Key, 내부 시스템명과 같은 민감한 정보는 Claude Code 입력에 포함하지 않습니다. 교육용 예제나 익명화된 구조를 기준으로 리뷰해야 합니다. 사내 PoC 전환 시에도 이 원칙은 동일하게 적용됩니다.

원칙 4 — 변경 결과는 사람이 직접 확인한다

Claude Code가 제안한 내용은 변경 전후 차이, 실행 명령, 테스트 결과를 사람이 확인한 뒤 반영하는 것이 안전합니다. AI 도구의 제안은 설계 리뷰 초안으로 받아들이고, 최종 판단은 항상 사람이 합니다. 제조 현업에서는 작은 수정이라도 권한, 오류 처리, 실행 기록 기준에 영향을 줄 수 있습니다.



구조를 이해하지 않은 상태에서 바로 수정을 요청하면, 기존 실습 흐름과 맞지 않는 방향으로 파일 구조나 함수 구조가 바뀔 수 있습니다. Claude Code는 통제 가능한 AI-assisted development 도구로 사용해야 합니다.

실습 1: Claude Code로 src/day3 구조 리뷰하기

이 실습의 목적은 Claude Code를 사용해 src/day3 폴더의 구조를 Agent 아키텍처 관점으로 이해하는 것입니다. 코드를 수정하거나 실행하는 것이 목적이 아닙니다. Claude Code에는 반드시 "코드를 수정하지 말고 구조 리뷰만 수행하라"고 명확히 요청해야 합니다.

파일명	역할 설명	아키텍처 계층
postgres_db_tool.py	제조 DB 및 설비 로그 조회 Tool. 반복 알람 이력, 온도·압력 이상 데이터 조회를 담당합니다.	DB Tool (근거 데이터)
search_manual_tool.py	기술 문서 검색 RAG Tool wrapper. 장애 대응 가이드, 정비 이력 문서 검색을 담당합니다.	RAG Tool (문서 근거)
tool_registry_fallback.py	get_equipment_overview와 search_manual 중심의 단순 fallback 예제입니다. 전체 Tool을 대체하는 구조가 아닙니다.	Fallback Registry
multi_agent_roles.py	Agent별 역할 정의와 State 전달 구조를 보여줍니다. 설비 담당, 공정 담당, 품질 담당 Agent 간 분업 구조를 확인합니다.	Multi-Agent 역할
day3_mcp_tool_agent_v2.py	Day3 통합 실행 launcher. MCP Tool 호출과 Multi-Agent 흐름을 통합 실행합니다.	통합 실행

실습 1: 리뷰 관점 및 실행 명령

리뷰 관점 — 설계 구조 확인 항목

- 각 파일이 담당하는 아키텍처 계층은 무엇인가
- MCP Tool 분할 기준이 업무 단위로 적절히 나뉘어 있는가
- Tool 입력·출력 구조가 표준화되어 있는가
- Multi-Agent 간 State 전달 구조가 명확한가
- LangGraph Routing 흐름에서 Tool 호출 조건이 설계되어 있는가
- State, agent_steps, db_results, rag_results 실행 기록 항목이 적절한가
- 현업 적용 시 수정해야 할 지점은 어디인가

반복 알람 분석 Agent 기준으로 보면

어떤 파일에서 DB 조회를 수행하고, 어떤 파일에서 기술 문서를 검색하며, 어떤 흐름에서 State가 전달되고 결과가 저장되는지 확인하는 방식으로 리뷰합니다.

예를 들어 온도 상승 알람이 반복 발생하는 경우, postgres_db_tool.py가 설비 로그에서 알람 이력을 조회하고, search_manual_tool.py가 기술 문서에서 관련 대응 가이드를 검색하며, multi_agent_roles.py가 설비 담당과 품질 담당 Agent 간 State를 전달하는 구조를 확인합니다.



Claude Code의 답변은 최종 정답이 아니라 설계 리뷰 초안입니다. Tool 분할과 State 전달 구조가 적절한지는 수강생이 다시 판단해야 합니다.

선택 실행 명령 (Windows PowerShell · uv 기준)

```
uv run python src/day3/postgres_db_tool.py
uv run python src/day3/search_manual_tool.py
uv run streamlit run src/day3/mcp_tool_client_streamlit_app.py
```

실습 2: 제한 수정 요청 프롬프트 작성하기

1 수정 요청 전 — 조건을 먼저 명시한다

함수명·클래스명·입출력 구조·파일 구조를 유지하고, 최소 변경안만 제안하도록 조건을 명시합니다.

2 수정 전 — 변경 위치와 이유를 먼저 설명하게 한다

어디를 왜 바꾸는지 먼저 설명하게 하고, 승인 전에는 코드를 작성하지 않도록 요청합니다.

3 수정 후 — 실행 명령과 테스트 케이스를 함께 요청한다

실행 명령과 테스트 케이스를 함께 요청해 변경 결과를 검증합니다.

3장 정리: Claude Code를 안전하게 활용하는 기준

이 장은 Claude Code를 3일차 수업에서 어떻게 활용할지 정리하는 장입니다. Claude Code는 전체 코드를 대신 완성하는 도구가 아닙니다. 프로젝트 구조를 이해하고 MCP Tool 설계와 Multi-Agent 흐름을 리뷰하는 보조 도구입니다. 이 차이를 명확히 유지하는 것이 안전한 활용의 출발점입니다.

구조 이해

src/day3 폴더의 주요 파일 역할을 Agent 아키텍처 관점으로 이해합니다. Tool 분할, Tool 설명-입출력 구조, LangGraph Routing, State 전달, 실행 기록 관점에서 검토합니다.

민감 정보 보호

실제 현업에서는 Claude Code에 사내 데이터, API Key, 내부 시스템명을 입력하지 않습니다. 교육용 예제나 익명화된 구조를 기준으로 검토하는 것이 안전합니다.

최소 변경 원칙

코드 수정이 필요한 경우에도 기존 구조를 유지한 상태에서 최소 변경안을 요청합니다. 변경 전후 차이와 실행 결과를 반드시 사람이 확인합니다.

현업 적용 판단

설비 상태 조회 Tool, 알람 이력 조회 Tool, 기술 문서 검색 Tool, State 전달 항목, 실행 기록 항목이 현업 적용 기준에 맞는지는 사람이 판단합니다.

3일차 실습 파일 역할 요약

파일	설계 역할
postgres_db_tool.py	제조 DB / 설비 로그 조회 Tool
search_manual_tool.py	기술 문서 검색 RAG Tool
tool_registry_fallback.py	단순 fallback 예제 (부분 대체)
multi_agent_roles.py	Multi-Agent 역할 및 State 전달
day3_mcp_tool_agent_v2.py	Day3 통합 실행 launcher

4일차 평가 및 5일차 PoC 전환과의 연결

다음 장에서는 API 직접 호출 방식과 MCP 기반 Tool 호출 방식의 차이를 비교하며, 왜 Tool Contract와 표준 호출 구조가 필요한지 살펴봅니다.

Tool Contract는 특정 JSON 파일 하나가 아닙니다. Tool 이름, 설명, 입력값, 출력 구조, 오류 처리 기준, 검증 규칙을 포함하는 설계 규칙 전체를 의미합니다. 4일차 평가에서는 이 기준으로 Tool 설계의 적절성을 판단하게 됩니다.

5일차 사내 PoC 전환 시에는 교육용 구조를 실제 현업 구조로 어떻게 대체할지 판단해야 합니다. 오늘 확인한 파일 구조와 설계 기준이 그 판단의 기반이 됩니다.



오늘 실습의 핵심은 Claude Code를 통해 가능한 AI-assisted development 도구로 사용하는 방법을 익히는 것입니다. 내부 시스템을 Agent Tool로 감싸는 구조를 이해하고, LangGraph 관점에서 Tool 호출 조건과 흐름을 설계하며, State와 agent_steps를 실행 기록 관점으로 확인하는 것이 3일차의 핵심 학습 목표입니다.